

Парадигма обмена данными между слоями Mozart ERP (версия 2.0)

Основные принципы

1. Каждый слой знает только о следующем.
 2. Никто не знает, что находится дальше его соседа.
 3. Данные на L3 — всегда типизированные объекты.
 4. Кэширование — на всех уровнях.
 5. Конфликты версий обрабатываются явно (Optimistic Locking).
-

Иерархия слоёв

Слой	Знает	Не знает
L3 (Клиент)	Адрес L2, протокол, команды	БД, SQL, схемы
L2 (Сервер)	DataSource, бизнес-логику	Детали реализации DataSource
DataSource	Свой источник (БД, кэш, API)	Всё остальное

Компоненты на L3

1. Раздельные сервисы (вместо одного толстого моста)

python

Не один класс, а несколько специализированных

```
class QueryService:
    """Только чтение данных."""
    def get_list(entity_alias, filters, order_by, page, page_size) -> ResultSet
    def get_item(entity_alias, instance_id) -> TypedEntity
    def get_reference_list(entity_alias, search_text, limit) ->
list[ReferenceItem]

class CommandService:
    """Изменение данных."""
    def save_item(entity_alias, data, version_id) -> SaveResult
    def delete_item(entity_alias, instance_id, version_id) -> bool

class FileService:
    """Работа с файлами."""
    def upload_file(entity_alias, instance_id, field_name, file_data, file_name)
-> str
    def download_file(file_id) -> bytes

class CalculationService:
    """Вызов расчётов и обработчиков."""
    def call(calc_alias, params, context) -> dict
```

Все сервисы используют один сетевой слой, но отвечают за разное.

2. Типизированные объекты (вместо сырых словарей)

L3 получает от L2 не JSON, а типизированные объекты.

python

Пример: объект "Персона" на L3

```
@dataclass
class PersonEntity:
    instance_id: int
    version_id: int
    cname: str
    clastname: str
    cfirstname: str
    birthdate: date | None
    inn: str | None
    is_active: bool

    def to_dict(self) -> dict:
        """Для отправки обратно на L2."""
        return asdict(self)

    @classmethod
    def from_dict(cls, data: dict) -> 'PersonEntity':
        return cls(**data)
```

L2 возвращает PersonEntity, а не {"id": 123, "cname": "Иванов"}. Формы работают с объектами, знают их структуру.

3. Кэширование на L3

python

```
class CacheManager:
    """Кэш для часто используемых данных."""

    def __init__(self):
        self._cache = {}
        self._ttl = {}

    def get(self, key: str, ttl_seconds: int = 300):
        """Вернуть из кэша, если не протух."""
        if key in self._cache:
            if time.time() < self._ttl.get(key, 0):
                return self._cache[key]
        return None

    def set(self, key: str, value, ttl_seconds: int = 300):
        """Положить в кэш на TTL."""
        self._cache[key] = value
        self._ttl[key] = time.time() + ttl_seconds

    def invalidate(self, pattern: str = None):
        """Сбросить кэш (по префиксу или весь)."""
        if pattern:
```

```
        keys = [k for k in self._cache.keys() if k.startswith(pattern)]
        for k in keys:
            del self._cache[k]
    else:
        self._cache.clear()
```

Что кэшируем на L3:

- Справочники (типы документов, роли, статусы) — TTL 300 сек
- Список БЕ для текущего пользователя — TTL 600 сек
- Результаты поиска — TTL 60 сек

Что НЕ кэшируем:

- Конкретные записи при редактировании (риск конфликтов)
 - Данные с версионированием
-

Компоненты на L2

1. DataSource как абстракция

python

```
class DataSource(ABC):
    """Абстрактный провайдер данных."""

    @abstractmethod
    def get_list(self, entity_alias: str, filters: dict,
                 order_by: str, page: int, page_size: int) -> ResultSet:
        pass

    @abstractmethod
    def get_item(self, entity_alias: str, instance_id: int) -> dict:
        pass

    @abstractmethod
    def save_item(self, entity_alias: str, data: dict, version_id: int) ->
    SaveResult:
        pass

    @abstractmethod
    def delete_item(self, entity_alias: str, instance_id: int, version_id: int)
    -> bool:
        pass
```

Реализации:

- PostgresDataSource — знает про БД, формирует SQL
 - MockDataSource — тестовые данные
 - CachedDataSource — обёртка с кэшированием на L2
 - RemoteDataSource — для распределённых инсталляций
-

2. ResultSet — стандартный формат ответа

python

```
@dataclass
class ResultSet:
    """Результат запроса списка."""
    total_count: int      # общее количество записей (без учёта LIMIT)
    page: int             # текущая страница
    page_size: int        # размер страницы
    rows: list[dict]      # строки (сырые словари)
    fields: list[str]     # какие поля возвращены
```

L3 получает ResultSet и знает, сколько всего записей для пагинации.

3. SaveResult — результат сохранения

python

```
@dataclass
class SaveResult:
    """Результат сохранения."""
    success: bool
    instance_id: int | None
    version_id: int | None # новая версия (если версионизируемая сущность)
    conflict: bool        # был ли конфликт версий
    message: str | None
```

Если конфликт — L3 должен показать пользователю выбор:

- Перезагрузить данные (с новым version_id) и показать изменения
 - Всё равно сохранить (принудительно, с риском потерять чужие изменения)
 - Отменить
-

Optimistic Locking — работа с версиями

Сценарий

1. L3 запрашивает персону: `get_item('person', 123)` → получает `PersonEntity(instance_id=123, version_id=5, ...)`
2. **Пользователь** редактирует форму (меняет фамилию)
3. Тем временем **другой пользователь** сохранил эту же персону → теперь в БД `version_id=6`
4. **Первый** нажимает «Сохранить», отправляет `save_item(..., version_id=5)`
5. L2 проверяет: `version_id` из запроса (5) не совпадает с текущим в БД (6)
6. L2 возвращает `SaveResult(conflict=True, version_id=6)`

7. **L3** показывает сообщение: «Запись была изменена другим пользователем. Перезагрузить?»
 8. **Пользователь** соглашается → L3 вызывает `get_item` снова → получает `version_id=6` и новые данные
 9. **L3** перезаполняет форму, пользователь делает свои правки поверх или отменяет
-

Протокол обмена (полная версия)

Команды L3 → L2

Команда	Параметры	Возвращает
<code>get_list</code>	<code>entity</code> , <code>filters</code> , <code>order_by</code> , <code>page</code> , <code>page_size</code>	<code>ResultSet</code>
<code>get_item</code>	<code>entity</code> , <code>instance_id</code>	<code>TypedEntity</code>
<code>save_item</code>	<code>entity</code> , <code>data</code> , <code>version_id</code>	<code>SaveResult</code>
<code>delete_item</code>	<code>entity</code> , <code>instance_id</code> , <code>version_id</code>	<code>bool</code>
<code>search_reference</code>	<code>entity</code> , <code>search_text</code> , <code>limit</code>	<code>list[ReferenceItem]</code>
<code>upload_file</code>	<code>entity</code> , <code>instance_id</code> , <code>field</code> , <code>file_data</code> , <code>file_name</code>	<code>FileInfo</code>
<code>download_file</code>	<code>file_id</code>	<code>bytes</code>
<code>call_calculation</code>	<code>calc_alias</code> , <code>params</code>	<code>dict</code>

Формат пакета

```
json
{
  "request_id": "uuid-1234",
  "command": "get_list",
  "payload": {
    "entity": "person",
    "filters": {"cversionstatus": "Active"},
    "order_by": "cname",
    "page": 1,
    "page_size": 50
  }
}
```

Формат ответа

```
json
{
  "request_id": "uuid-1234",
  "success": true,
  "payload": {
    "total_count": 150,
    "page": 1,
    "page_size": 50,
    "rows": [...],
    "fields": ["instance_id", "cname", "clastname"]
  },
  "error": null
}
```

```
}
```

Ошибка

json

```
{
  "request_id": "uuid-1234",
  "success": false,
  "payload": null,
  "error": {
    "code": "VERSION_CONFLICT",
    "message": "Record was modified by another user",
    "current_version": 6
  }
}
```

Кэширование на L2

L2 тоже может кэшировать частые запросы (но с учётом версионности).

python

```
class CachedDataSource(DataSource):
    """Обёртка, добавляющая кэш на L2."""

    def __init__(self, inner: DataSource, cache_ttl: int = 60):
        self._inner = inner
        self._cache = {}
        self._ttl = cache_ttl

    def get_list(self, entity_alias, filters, order_by, page, page_size):
        key = f"{entity_alias}:{hash(filters)}:{order_by}:{page}:{page_size}"
        if key in self._cache:
            return self._cache[key]
        result = self._inner.get_list(...)
        self._cache[key] = result
        return result

    def save_item(self, entity_alias, data, version_id):
        # При сохранении сбрасываем кэш списков для этой сущности
        self._invalidate_cache(entity_alias)
        return self._inner.save_item(entity_alias, data, version_id)
```

Что кэшируем на L2:

- Списки с пагинацией (но нужно сбрасывать при изменениях)
 - Статические справочники (почтовые индексы, БИК)
 - Результаты поиска (короткое TTL)
-

Итог: сравнительная таблица

Аспект

Версия 1.0

Версия 2.0 (дополненная)

Единый класс обмена	Толстый мост	Раздельные сервисы
Типизация данных	Сырые словари	Typed dataclass
Кэширование на L3	Нет	Есть (с TTL)
Optimistic Locking	Нет	Есть (version_id)
ResultSet	Нет	Есть (total_count, page, page_size)
SaveResult	Нет	Есть (success, conflict, new_version)
Кэширование на L2	Нет	Есть (опционально)
Формат ошибок	Неопределён	Единый с code и message

Что остаётся неизменным

1. L3 не знает про БД.
2. L2 не знает про реализацию DataSource.
3. Формы настраиваются в конфигураторе, а не в коде.
4. Протокол между L3 и L2 остаётся простым (JSON).
5. Никто, кроме DataSource, не формирует SQL.

Эта архитектура готова к масштабированию и реальной эксплуатации.